

Reducing Dataset Size in Power BI

1. Remove Unused Columns

- ❖ **Why?** Every column stored in the model consumes memory, even if not used in visuals.
- ❖ **How?**
 - During **Power Query (ETL)** → remove unnecessary columns (e.g., IDs, metadata, extra text fields).
 - Keep only columns required for reporting, calculations, or relationships.
- ❖ **Example:**
 - You import Customer table with 30 columns, but your report only needs CustomerID, Name, City. → Drop the rest.

2. Remove Unused Rows

- ❖ Filter data at the source (SQL, Power Query) instead of loading all history.
- ❖ Example: Instead of loading 10 years of sales, keep last 3 years for analysis.

3. Use Correct Data Types

- ❖ Change unnecessary **Text** → **Whole Number/Date/Boolean**.
- ❖ Smaller data types = less memory.
- ❖ Example: Don't keep "2025-08-24 12:34:56" if you only need Date.

4. Summarization / Aggregation

- Instead of loading **detailed transactional data**, pre-aggregate it at source.
- Example:
 - ❖ Instead of 1M rows of daily transactions, load **monthly totals**.
 - ❖ Keep detailed data only if required for drill-through.

5. Disable Auto Date/Time

- ❖ Power BI automatically creates hidden date tables for every date column (memory heavy).
- ❖ Turn off Auto Date/Time and create a **single calendar table** instead.

6. Reduce Cardinality

- ✓ High cardinality columns (unique IDs, transaction numbers, long text) increase size.
- ✓ Best practices:
 - ❖ Remove columns with unique values (like GUIDs, invoice numbers).
 - ❖ Use numeric keys instead of text for relationships.
 - ❖ Group/categorize where possible (e.g., instead of keeping all 1,000 product SKUs, use 20 product categories if that's enough).

7. Incremental Refresh

- ✓ Instead of reloading the full dataset daily, refresh only new/changed rows.
- ✓ Helps keep datasets smaller & refresh faster.

8. Use Measures instead of Calculated Columns

- ✓ Measures are computed on the fly and **don't add size** to the model.
- ✓ Prefer measures whenever possible.

Interview-ready Answer If asked “*How do you reduce dataset size in Power BI?*”, you can say:

“I start by removing unused columns and rows at the Power Query stage, use proper data types, and disable auto date/time. For large datasets, I prefer summarization or pre-aggregations at the source, reduce cardinality by removing unique IDs, and apply incremental refresh. I also prefer using measures instead of calculated columns to avoid bloating the model.”

Difference Between CALCULATE vs FILTER in Power BI (DAX)

1. CALCULATE

- ✓ **Definition:** The most powerful DAX function. It **changes the filter context** of a calculation.
- ✓ **Returns:** A **scalar value** (a number, text, or boolean) depending on the expression.
- ✓ **Usage:** Wraps an aggregation (SUM, AVERAGE, COUNT, etc.) and modifies the filters.
- ✓ **Example:**

Total Sales = SUM(Sales[Revenue])

```
Sales_2024 = CALCULATE(  
    SUM(Sales[Revenue]),  
    Sales[Year] = 2024  
)
```

Here, CALCULATE applies the condition *only for 2024* before summing.

2. FILTER

- Definition:** Returns a **table** with rows filtered according to a condition.
- Returns:** Always a **table** (not a number).
- Usage:** Used inside functions like CALCULATE, SUMX, AVERAGEX where a table input is needed.
- Example:**

Filtered_Table = FILTER(Sales, Sales[Year] = 2024)

```
Sales_2024 = CALCULATE(  
    SUM(Sales[Revenue]),  
    FILTER(Sales, Sales[Year] = 2024)  
)
```

Here, FILTER produces a table with only 2024 rows, then CALCULATE sums the revenue from that table.

Key Differences

Feature	CALCULATE	FILTER
Return type	Scalar (single value)	Table
Purpose	Modify filter context for an expression	Return subset of rows
Typical Use	Aggregations with conditions	Complex filtering (multiple conditions, row-level logic)
Performance	Faster for simple conditions	Slower (row-by-row evaluation)
Example	CALCULATE(SUM(Sales[Revenue]), Sales[Year]=2024)	FILTER(Sales, Sales[Year]=2024)

When to Use

- ✓ Use CALCULATE: When you want to compute a value under modified filter conditions (e.g., "Sales for 2024 only").
- ✓ Use FILTER: When you need complex filtering logic (multiple conditions, OR logic, row-by-row checks) — usually nested inside CALCULATE.

Interview Tip: If they ask “*Can FILTER be used alone?*” → Yes, but it just returns a table. To turn it into a number, you must wrap it with something like SUMX.

Static vs Dynamic RLS in Power BI

Static Row-Level Security (RLS)

Definition: Security rules are **hard-coded** (fixed filters).

How it works: You create multiple roles in Power BI Desktop → each role has a filter applied on a column.

Example:

- Role 1 → Region = "India"
- Role 2 → Region = "USA"
- Any user assigned to Role 1 will always see **only India data**.

[Region] = "India"

Pros: Easy to set up, quick for small datasets.

Cons: Not scalable — if you have 50 regions, you need 50 roles.

Dynamic Row-Level Security (RLS)

❖ **Definition:** Security rules are **user-dependent**, based on who logs in.

❖ **How it works:**

- ✓ Create a **mapping/security table** with User → Allowed Values.
- ✓ Use USERPRINCIPALNAME() (user's email/login) in DAX.

❖ **Example:**

◆ Security Table:

UserEmail	Region
rahul@company.com	India
priya@company.com	USA

DAX Filter Rule:

[Region] = LOOKUPVALUE(Security[Region], Security[UserEmail], USERPRINCIPALNAME())

- ❖ If Rahul logs in → he sees **India** data.
- ❖ If Priya logs in → she sees **USA** data.
- ❖ **Pros:** Scalable — one role can handle thousands of users.
- ❖ **Cons:** Needs extra setup (security table + DAX).

Key Differences

Feature	Static RLS	Dynamic RLS
Filter Type	Hard-coded	User-dependent (lookup with login)
Roles Needed	One per filter (many roles)	Usually one role (scalable)
Setup	Simple	Requires mapping table + DAX
Best For	Small teams, limited categories	Enterprises, multiple users/regions

Interview Answer

“Static RLS uses fixed filters like Region = "India" and works only for small datasets. Dynamic RLS is user-based, using functions like USERPRINCIPALNAME() with a mapping table, so each user only sees their own data. In real-world enterprise scenarios, dynamic RLS is preferred because it is scalable and easier to maintain.”